Brief Announcement: Stranger-Free Tasks

Eli Gafni
UCLA
Los Angeles, California
eli@ucla.edu

Giuliano Losa
Stellar Development Foundation
San Francisco, USA
giuliano@losa.fr

Michel Raynal
Univ. Rennes, Inria, CNRS, IRISA
Rennes, France
michel.raynal@irisa.fr

Gadi Taubenfeld
Reichman University
Herzliya, Israel
tgadi@runi.ac.il

Abstract

Delporte-Gallet et al. show that, in a system of n processes, it is both necessary and sufficient to use n multi-writer multi-reader (MWMR) registers, that are not pre-allocated, to emulate with non-blocking progress n single-writer multi-reader (SWMR) registers that are uniquely pre-allocated. They conclude with the significant result that n MWMR registers are sufficient to solve any task solvable read-write wait-free. However, they mistakenly claim—likely inadvertently—that n MWMR registers are also necessary to solve any task solvable read-write wait-free (a counterexample is the splitter task, which is solvable for any number of processes with just 2 MWMR registers).

We propose the new notion of stranger-free task where, roughly speaking, for every two processes, at least one must know about the other. We show that n MWMR registers are necessary to solve stranger-free tasks. However, there is an infinite hierarchy of tasks that are not stranger-free: for every integer $k > 0$, we exhibit a task that is solvable for an infinite number of processes using $k+1$ MWMR registers but that is not solvable for $k+1$ processes using k MWMR registers. It remains an open question whether there are tasks that are not stranger-free and yet their wait-free solution requires n MWMR registers.

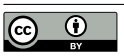
CCS Concepts

• **Theory of computation → Distributed algorithms**.

Keywords

Asynchrony, Read/write registers, Wait-freedom, Task, Covering argument, Lower bound, Space complexity, Stranger-freedom

ACM Reference Format:
Eli Gafni, Giuliano Losa, Michel Raynal, and Gadi Taubenfeld. 2025. Brief Announcement: Stranger-Free Tasks. In *ACM Symposium on Principles of Distributed Computing (PODC '25), June 16–20, 2025, Huatulco, Mexico.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3732772.3733545



This work is licensed under a Creative Commons Attribution 4.0 International License.
PODC '25, Huatulco, Mexico
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1885-4/2025/06
https://doi.org/10.1145/3732772.3733545

1 Introduction

By definition, a task for n processes is solvable read-write wait-free when there is a wait-free algorithm for n processes that solves the task using n single-writer multi-reader (SWMR) registers, where each register r_i is uniquely allocated to a process p_i, and p_i is the only process allowed to write to r_i.

If, instead, we have n multi-writer multi-reader (MWMR) registers that are not uniquely pre-allocated, it is a priori unclear whether we can still solve all read-write wait-free solvable tasks. This is because processes may write to the same registers and obliterate each other's writes. Delporte-Gallet et al. [6] show that every task solvable read-write wait-free is also solvable wait-free with n MWMR registers that are not pre-allocated. To prove this property, they present a non-blocking emulation of n pre-allocated SWMR registers using n MWMR registers.

But what if we only have $m < n$ registers available[1]? In this case, Delporte-Gallet et al. show that it is impossible to emulate n SWMR registers, which suggests that some tasks are not solvable even with $n-1$ MWMR registers.

Many tasks are nevertheless solvable with fewer registers than processes. For example, the well-known splitter task [11][2] is solvable wait-free for an unbounded number of processes using only two registers. Another example is the adopt-commit task [7], which is solvable wait-free using $\text{fact}^{-1}(n)+2 = O(\log n/\log\log n)$ registers [2] (fact^{-1} is the inverse of the factorial function).

What makes a task require n registers? Towards answering this question, we propose the new notion of stranger-freedom. A task is stranger-free when, for every read-write wait-free full-information algorithm that solves the task, for every two participating processes p_i and p_j, either p_i appears in p_j's final view or p_j appears in p_i's final view. For example, it is easy to see that lattice agreement [3], in the lattice of sets of process identifiers, is a stranger-free task. We show that every stranger-free task requires n registers.

Obviously, not all tasks are stranger-free. Since solving stranger-free tasks wait-free requires n registers, tasks solvable wait-free with $m < n$ registers are not stranger-free. We show that there is an infinite hierarchy of non-stranger-free tasks: for every integer $k > 0$, we exhibit a task that is solvable for an infinite number of processes using $k+1$ registers (even when registers are anonymous [12]) but that is not solvable for $k+1$ processes using k registers.

[1] With fewer registers than processes, the registers must be MWMR registers, and we will assume so throughout the paper
[2] The basic mechanism implementing a splitter was introduced by Lamport [10]

We do not know whether there are tasks that are not stranger-free and yet whose wait-free solution requires n registers. An example may be renaming in $n(n+1)/2$ names. The renaming algorithm of Moir and Anderson [11] renames n processes into $n(n+1)/2$ names using a network of splitter objects. In this algorithm, it is possible that two processes, that take different directions at some splitter, remain complete strangers even if we make it a full-information algorithm. Renaming in $n(n+1)/2$ names is therefore not stranger-free. It remains to determine if n registers is a lower bound for this instance of renaming.

2 Stranger-free tasks

We consider the usual notion of an asynchronous, read-write wait-free algorithm solving a task. To define stranger-free tasks, we first need to define the full-information version of an algorithm.

Full-information algorithms. A full-information algorithm is an algorithm in which each process p always writes a tuple, which we call p's view, consisting of p's identifier (if it has one), p's input value, and the full sequence of reads that p obtained so far. Thus, a full-information algorithm only specifies which register to read or write, and when and what to output.

Full-information version of an algorithm. Given a read-write algorithm $\mathcal{A}$, the full-information version of $\mathcal{A}$ is the full-information algorithm such that, to determine its next step, each process uses its view to simulate what its next step would be in $\mathcal{A}$, and then writes or reads the same register as $\mathcal{A}$ would; moreover, if it produces an output in $\mathcal{A}$, then it produces the same output.

Note that an algorithm and its full-information version solve the same tasks and use the same number of registers.

Stranger-freedom. A task is *stranger-free* when, for every full-information algorithm that solves the task wait-free using read-write registers only, for every execution of the algorithm, for every two processes i and j, after both i and j output, either i appears in j's view or j appears in i's view. We say that two processes are *complete strangers* when neither appears in the other's view. Thus, satisfying stranger-freedom means that no two processes that output are complete strangers.

Theorem 2.1. *For every $n > 1$ and for every stranger-free task for n processes, if $\mathcal{A}$ is a wait-free algorithm that solves the task using registers only, then $\mathcal{A}$ uses at least n registers.*

Proof. The proof follows a long line of covering arguments, starting with the pioneering work of Burns and Lynch [4].

Let us denote the n processes $p_1, ..., p_n$. Like Burns and Lynch, we say that a process p_i covers a register r in a given run when the next step by p_i in that run is writing r. Consider a full-information algorithm $\mathcal{A}$ solving the task. We prove by induction (on the number of covered registers j) that, for every $1 \leq j \leq n$, there is a run ρ_j of $\mathcal{A}$ such that in ρ_j (1) only processes $p_1, ..., p_j$ participate, (2) j different registers are covered, and (3) p_1 has not yet been written in any register. In particular, in the ρ_n run, n different registers are covered, which implies that $\mathcal{A}$ must use at least n registers.

This implies that any algorithm solving the task must use at least n registers, or otherwise its full-information version would solve the task with fewer than n registers.

Base of the induction: $j = 1$. We show that when p_1 runs solo (i.e., alone), it must eventually write into some shared register. The run ρ_1 consists of p_1 running solo until its next step is to write. When p_1 runs solo, by definition, p_1 will eventually terminate, and since p_1 cannot know about any other process except itself, its final view, V_1, cannot include the identifier of any other process. After p_1 terminates, we let p_2 run until it also terminates. If p_1 has not written before it terminates, then p_2 cannot know that p_1 was active, which means that p_2's view V_2 does not include the identifier of p_1. Thus, in such a case (where p_1 does not write), p_1 and p_2 would be complete strangers, which contradicts the assumption that $\mathcal{A}$ satisfies stranger-freedom. Hence, we conclude that when p_1 runs solo, it will eventually try to write in some register and, as already explained, ρ_1 is the run just before p_1 executes its first write. This completes the base of the induction.

The induction hypothesis. Consider j such that $1 \leq j \leq n-1$ and there is a run ρ_j of $\mathcal{A}$ such that in ρ_j (1) only the processes p_1 to p_j participate, (2) j different registers are covered, (3) p_1 has not been written into any register yet.

The induction step. We prove that there is a finite run ρ_{j+1} of $\mathcal{A}$ such that in ρ_{j+1} (1) only the processes $p_1, ..., p_{j+1}$ participate, (2) $j+1$ different registers are covered, (3) p_1 has not been written into any register yet.

By the induction hypothesis, there exists a run ρ_j of $\mathcal{A}$ such that in ρ_j (1) only the processes $p_1, ..., p_j$ participate, (2) j different registers are covered, (3) p_1 has not been written into any register yet. Let R denote the set of the j different registers covered in ρ_j.

Let us extend the run ρ_j with steps by process p_{j+1} only, and let p_{j+1} run until it terminates; let δ_1 be this run. Next, we prove that before p_{j+1} terminates, p_{j+1} must write into some register that is not in R. The run ρ_{j+1} (in which $j+1$ different registers are covered) is the run that extends ρ_j until the point just before p_{j+1} executes its first write into a register not in R.

Since p_1 has not written yet in δ_1, p_{j+1} cannot know that p_1 is active, which implies that V_{j+1}, the final view of p_{j+1}, does not include the identifier of p_1. Now, we extend δ_1 by first letting each one of the j processes execute one step. This causes all the j registers in R to be overwritten. After that, we let all the processes run until they all terminate. Let us call the resulting run δ_2.

Suppose p_{j+1} terminates without writing into a register that is not in R. In that case, when the j other processes overwrite all the j registers in R, they have erased any trace that p_{j+1} has participated. That is, the run δ_2 would be indistinguishable (for all the j processes except p_{j+1}) from the same run in which all the steps of p_{j+1} are removed. Thus, in δ_2, p_1 cannot know that p_{j+1} was active, which means that p_1's view V_1 does not include the identifier of p_{j+1}. Thus, in such a case (where p_{j+1} does not write into a register not in R), p_1 and p_{j+1} would be complete strangers, which contradicts the assumption that $\mathcal{A}$ satisfies stranger-freedom.

Thus, we must conclude that before p_{j+1} terminates, it has to write into a register not in R, and we can now construct the run ρ_{j+1} as explained above. This completes the induction step.

We have shown that, for every $1 \leq j \leq n$, there exists a run ρ_j of $\mathcal{A}$, in which j registers are covered. In particular, taking $j = n$, there exists a run ρ_n of $\mathcal{A}$, in which n registers are covered. This implies that $\mathcal{A}$ uses at least n (different) registers. □

Notice that the theorem is still valid when we replace wait-freedom with obstruction-freedom.

3 An Infinite Non-Stranger-Free Hierarchy

Theorem 3.1. *For every $k \geq 1$, there exists a task that can be solved wait-free using only $k+1$ registers for infinitely many processes but cannot be solved using k registers even for $k+1$ processes.*

We show this theorem by exhibiting such a task.

Definition. For every $k \geq 1$, we define the *k-bounded lattice-agreement* task as follows. Processes receive no input, and each participating process outputs a set of process identifiers such that:

- *Validity.* Each participating process outputs a set of participating processes.
- *Self-inclusion.* Each participating process outputs a set of processes that contains itself.
- *Containment.* Every two outputs of cardinality at most k must be related by containment.

Let us note that, for $k > 1$, any algorithm that solves k-bounded lattice-agreement also solves $(k-1)$-bounded lattice-agreement, but not necessarily vice versa.

3.1 Upper Bound: $k+1$ Registers are Sufficient

This section presents, for every $k \geq 1$, a wait-free algorithm that solves k-bounded lattice-agreement for infinitely many processes using $k+1$ registers. Pseudocode appears in Algorithm 1.

In this algorithm, processes communicate using a shared array A consisting of $k+1$ registers. Each process p keeps track of the set of processes it has seen so far (local variable *seen*), and it eventually outputs this set. Until p determines it can output, it executes a loop in which it first writes to some position of the shared array A (line 9, going round-robin), and then tries to take an atomic snapshot of the array. When p writes to a position in the array (line 9), it writes a triple, unique among all processes, consisting of its identifier, a sequence number, and the set of processes it has seen so far. To try to take a snapshot (lines 12 to 22), p uses the double-collect idea of Afek et al. [1]: because the triples written are unique, two identical reads of the whole array guarantee nothing changed in between, and thus they form an atomic snapshot.

At the end of each loop iteration (lines 23 to 26), p outputs and terminates if either it has seen strictly more than k processes, or it successfully took a snapshot of the array A and all entries of A contain exactly the set of processes that p has seen so far. Otherwise, p goes back to line 8 for a new loop iteration.

The safety of the algorithm relies on the fact that, once a process p obtains a snapshot in which every register in the shared array contains the set of processes that p has seen so far, then p can be sure that, unless at least $k+1$ processes participate, the set will always be a subset of at least one register in the shared array. That is because k processes are not enough to obliterate all $k+1$ registers before they have to read the whole array. A similar reasoning is used by Delporte-Gallet et al. [5] in their snapshot algorithm.

Wait-freedom is ensured, even with infinitely many processes, because processes that keep interfering and preventing each other's snapshots from succeeding have *seen* sets that converge to the same value or eventually grow in cardinality past k.

Algorithm 1 k-bounded lattice agreement, code for process p

```
1:  shared A[r] ← ⟨0, 0, ∅⟩ for all r ∈ 1..k+1
2:  local output ← ∅
3:  local seen ← {p}                          ▷ Set of processes seen so far
4:  local seq ← 0                             ▷ A sequence number
5:  local i ← 1                               ▷ Iteration variable
6:  local snapshot ← true                     ▷ Boolean
7:  local collect[r] ← ⊥ for all r ∈ 1..k+1   ▷ Stores reads of A
8:  while output = ∅ do
9:      A[i] ← ⟨p, seq, seen⟩
10:     i ← if (i ≤ k) then i + 1 else 1
11:     seq ← seq + 1
12:     for j = 1 to k + 1 do
13:         collect[j] ← A[j]
14:         seen ← seen ∪ collect[j][3]
15:     end for
16:     snapshot ← true
17:     for j = 1 to k + 1 do
18:         if A[j] ≠ collect[j] then
19:             snapshot ← false
20:         end if
21:         seen ← seen ∪ collect[j][3]
22:     end for
23:     if |seen| > k then
24:         output ← seen
25:     else if snapshot ∧ ∀r ∈ 1..k+1 : seen = collect[r][3] then
26:         output ← seen
27:     end if                                ▷ Go back to line 8
28: end while
```

Finally, note that this algorithm works even if the shared memory is anonymous [12] (the order in which each process iterates over the array positions does not matter). If, additionally, processes are anonymous, then they cannot include any identifier in their write at line 9, which means that two processes may write the same tuple, and thus two identical collects do not guarantee a snapshot anymore. As proposed by Guerraoui and Ruppert [8, Proposition 4][3], if we assume a known number of processes n, we can fix this problem by collecting $k(n-1)+2$ times instead of just twice. This gives us a fully-anonymous, wait-free k-bounded lattice agreement algorithm for n processes using $k+1$ registers.

3.2 Lower Bound: $k+1$ Registers are Necessary

To prove this result, first observe that k-bounded lattice-agreement for $k+1$ processes ensures that every pair of outputs are related by containment. This is because, by the containment property, all outputs of cardinality at most k are related by containment, and there are only $k+1$ processes. With the self-inclusion property, we conclude that, for every two processes, at least one of them produces an output that contains the other, and thus k-bounded lattice-agreement for $k+1$ processes is stranger-free. We conclude using Theorem 2.1 that no wait-free algorithm solves k-bounded lattice-agreement for $k+1$ processes using k registers or less.

[3]The proof of the proposition, which contains the algorithm we are interested in, does not appear in the peer-reviewed version [9] or their work

References

[1] Yehuda Afek, Hagit Attiya, Danny Dolev, Eli Gafni, Michael Merritt, and Nir Shavit. 1993. Atomic Snapshots of Shared Memory. *J. ACM* 40, 4 (Sept. 1993), 873–890. https://doi.org/10.1145/153724.153741

[2] James Aspnes and Faith Ellen. 2014. Tight Bounds for Adopt-Commit Objects. *Theory of Computing Systems* 55, 3 (Oct. 2014), 451–474. https://doi.org/10.1007/s00224-013-9448-1

[3] Hagit Attiya, Maurice Herlihy, and Ophir Rachman. 1995. Atomic Snapshots Using Lattice Agreement. *Distributed Computing* 8, 3 (March 1995), 121–132. https://doi.org/10.1007/BF02242714

[4] J. E. Burns and N. A. Lynch. 1993. Bounds on Shared Memory for Mutual Exclusion. *Information and Computation* 107, 2 (Dec. 1993), 171–184. https://doi.org/10.1006/inco.1993.1065

[5] Carole Delporte-Gallet, Hugues Fauconnier, Eli Gafni, and Leslie Lamport. 2013. Adaptive Register Allocation with a Linear Number of Registers. In *International Symposium on Distributed Computing*. Springer, 269–283.

[6] Carole Delporte-Gallet, Hugues Fauconnier, Eli Gafni, and Sergio Rajsbaum. 2013. Linear Space Bootstrap Communication Schemes. In *Distributed Computing and Networking*, Davide Frey, Michel Raynal, Saswati Sarkar, Rudrapatna K. Shyamasundar, and Prasun Sinha (Eds.). Springer, Berlin, Heidelberg, 363–377. https://doi.org/10.1007/978-3-642-35668-1_25

[7] Eli Gafni. 1998. Round-by-Round Fault Detectors (Extended Abstract): Unifying Synchrony and Asynchrony. In *Proceedings of the Seventeenth Annual ACM Symposium on Principles of Distributed Computing (PODC '98)*. ACM, New York, NY, USA, 143–152. https://doi.org/10.1145/277697.277724

[8] Rachid Guerraoui and Eric Ruppert. 2004. *What Can Be Implemented Anonymously?* Technical Report. https://infoscience.epfl.ch/handle/20.500.14299/214730

[9] Rachid Guerraoui and Eric Ruppert. 2005. What Can Be Implemented Anonymously?. In *Distributed Computing (Lecture Notes in Computer Science)*, Pierre Fraigniaud (Ed.). Springer, Berlin, Heidelberg, 244–259. https://doi.org/10.1007/11561927_19

[10] Leslie Lamport. 1987. A Fast Mutual Exclusion Algorithm. *ACM Trans. Comput. Syst.* 5, 1 (Jan. 1987), 1–11. https://doi.org/10.1145/7351.7352

[11] Mark Moir and James H. Anderson. 1995. Wait-Free Algorithms for Fast, Long-Lived Renaming. *Science of Computer Programming* 25, 1 (Oct. 1995), 1–39. https://doi.org/10.1016/0167-6423(95)00009-H

[12] Gadi Taubenfeld. 2022. Anonymous Shared Memory. *J. ACM* 69, 4 (Aug. 2022), 24:1–24:30. https://doi.org/10.1145/3529752